QMWS Spring 2026: Introduction to Python for Data Analysis

Day One: Foundations

Gavin Klorfine & Deborah Laze

Section 1

Installation

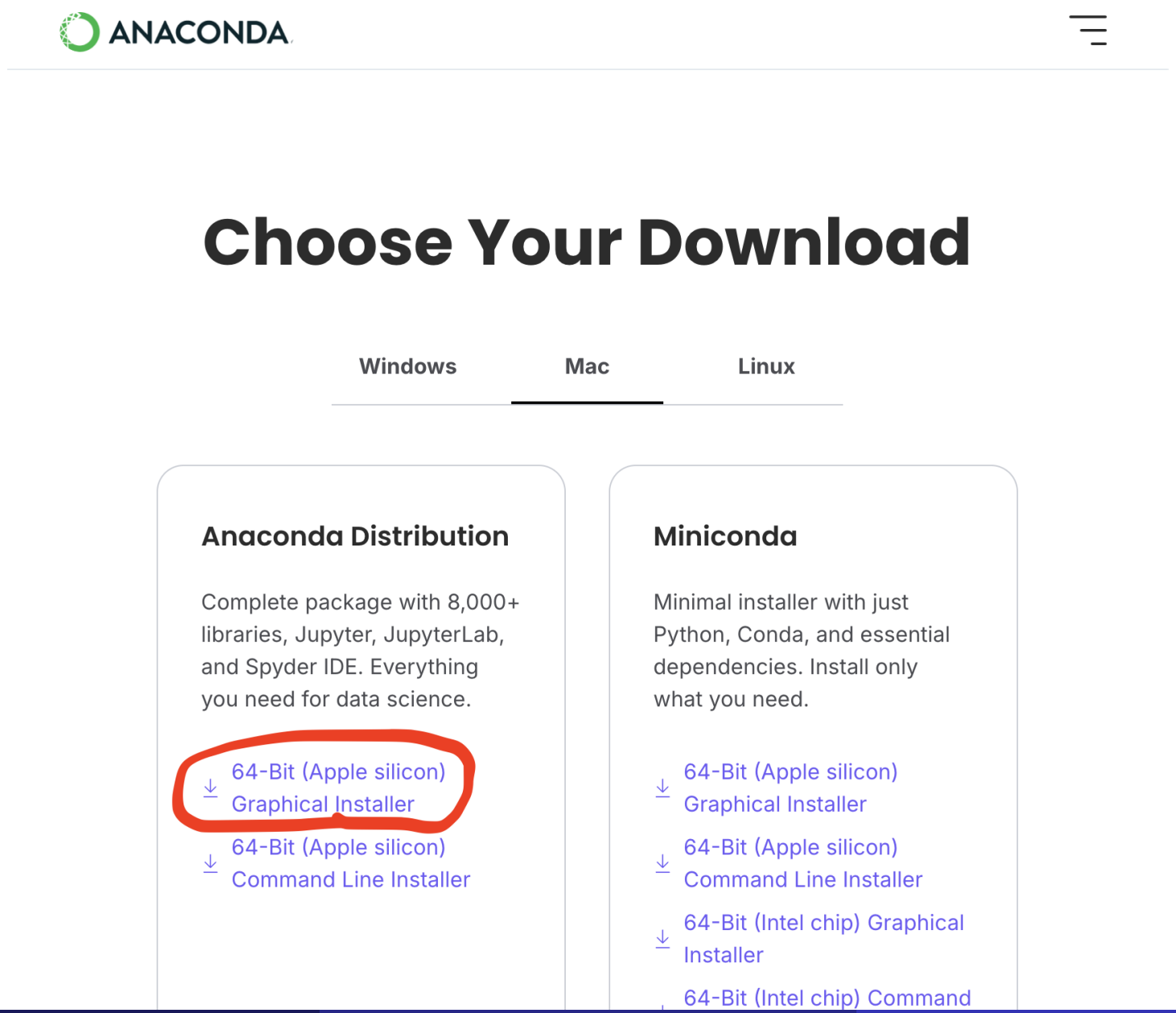
Anaconda

- A tool for configuring Python **environments**
  - More often than not, you need to install **packages** that allow you to code more easily/efficiently
  - Allows for version control
  - Issues can arise if environments are not used, for example:
    - If you install two packages that conflict with one another
    - If you install a package that requires a specific version of Python
  - Anaconda allows these issues to be isolated to a specific environment, rather than affecting your whole Python installation
    - Especially useful if you have more than one project going on

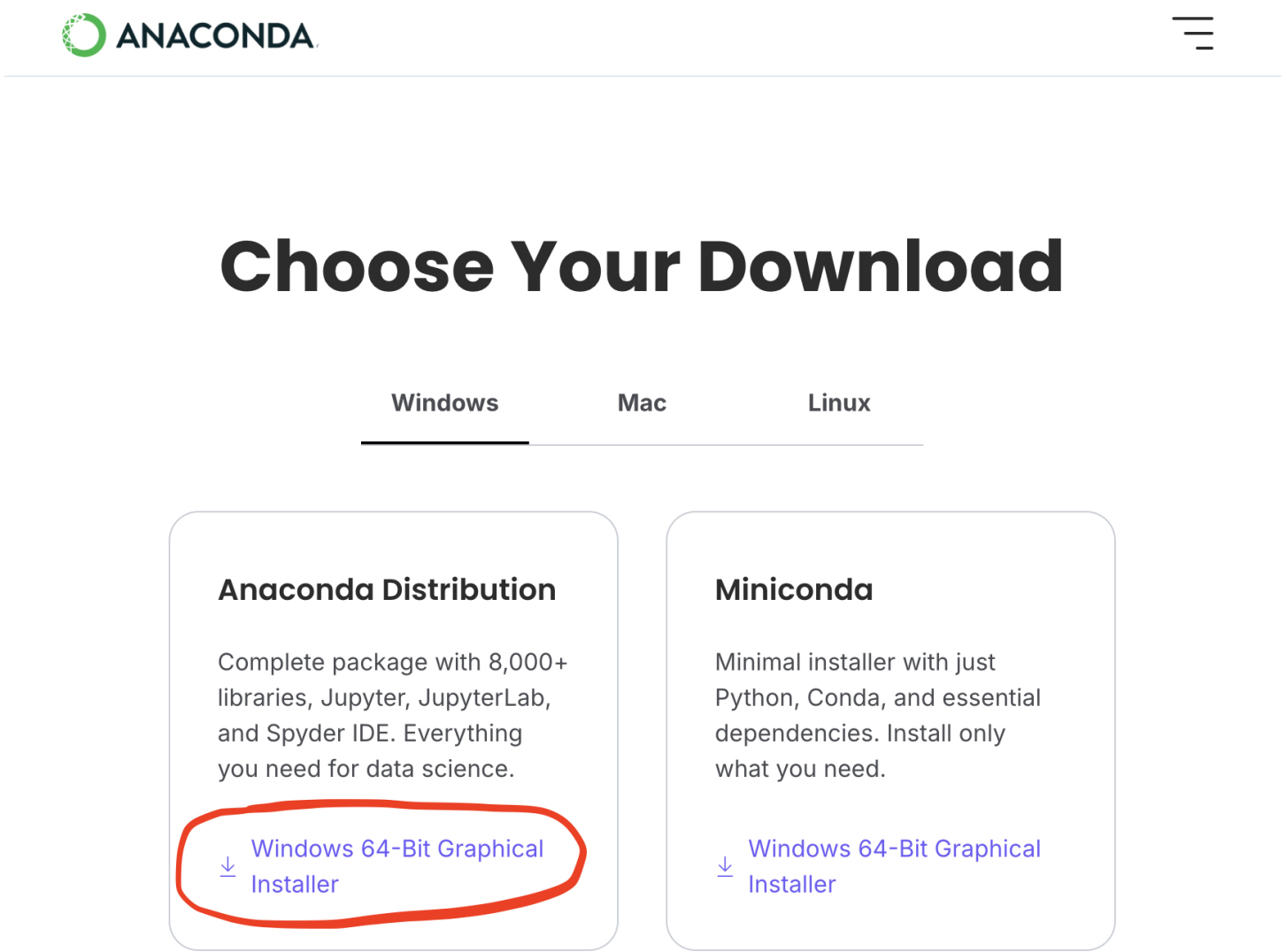
Anaconda

- Installation link: https://www.anaconda.com/download
- The simplest way to use Anaconda is via the **Navigator**
- To install:
  1. Make an account
  2. Install the full distribution via the "Graphical Installer"

- Installation link: https://www.anaconda.com/download
- On Mac:



- Installation link: https://www.anaconda.com/download
- On Windows:

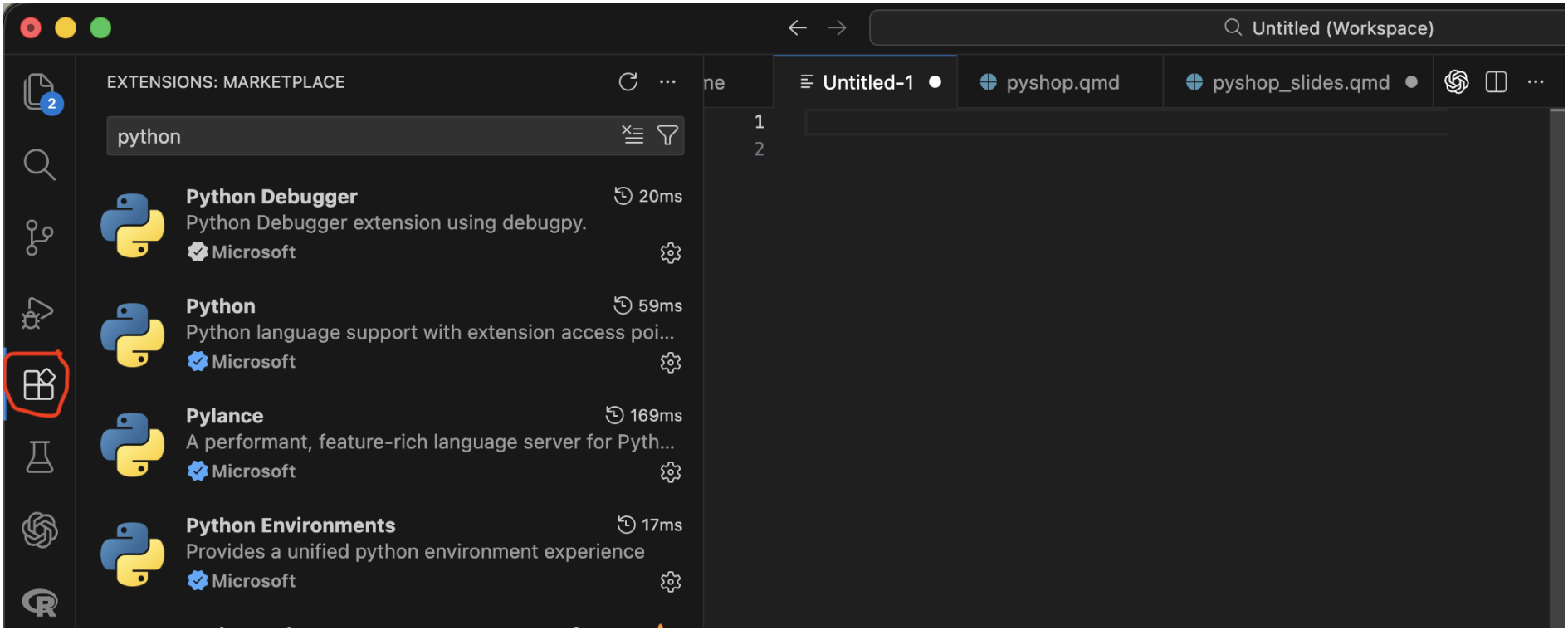


Visual Studio (VS) Code

- Installation link: https://code.visualstudio.com/download
- Integrated Development Environment (IDE)
- Can use it to code in many different languages
- Commonly used in both academia and industry

Visual Studio (VS) Code

- Once installed, go to the "Extensions" tab and search "Python"
- Install the below extensions:



Stop here!

If you:

1. Are able to open the **Anaconda Navigator**
2. Have installed **VS Code and the listed extensions**

You are ready for the workshop! We will cover next steps at the beginning of the first session.

Else, please email either of us for help:

- Gavin (gklorfin@yorku.ca)
- Bora (laze@yorku.ca)

Setting Up a Conda Environment

- See the presenter's shared screen

Visual Studio (VS) Code

- To point VS Code to your new Conda environment:
  1. Open the Command Palette
     a. `Ctrl + Shift + P` on Windows
     b. `Command + Shift + P` on MacOS
  2. Search "Python: Select Interpreter"
  3. Select the interpreter that corresponds to your new environment (the name of the environment will be in parentheses)

Using Python

Scripts

- To create a new Python script (a file with a `.py` extension):
  1. File -> New File -> Python File
- These scripts are generally executed from top to bottom, and are useful for writing longer chunks of code
- You can run a script with the play button in the top right corner of VS Code

Using Python

Scripts

- To create a new Python script (a file with a `.py` extension):
  1. File -> New File -> Python File
- These scripts are generally executed from top to bottom, and are useful for writing longer chunks of code
- You can run a script with the play button in the top right corner of VS Code

Console

- The console is useful for executing smaller bits of code
- To open the console:
  1. Open the Command Palette -> search "Python: Start Terminal REPL"
- Type in a line of code and hit `enter` to execute it
  - Multiple lines may be typed by using `shift + enter`

Section 2

The Basics

Variables

- In programming, a variable is a space in computer memory where values may be stored

Defining and Printing Variables

```
x = 67

print(x)
```

```
67
```

Variables

- Variable names should be short but descriptive (e.g., a variable for the mean of `x` should be named `mean_x`)
  - We are using `x` for pedagogical purposes... In practice, `x` is likely non-descriptive!

Variables

- Variable names should be short but descriptive (e.g., a variable for the mean of `x` should be named `mean_x`)
  - We are using `x` for pedagogical purposes... In practice, `x` is likely non-descriptive!
- When a variable name contains multiple words (e.g., apple size), there are a couple of approaches:
  - Use an underscore (e.g., `apple_size`)
  - Use "camel case" (e.g., `appleSize`)
    - Camel case has the first word all lowercase, with subsequent words having their first letter capitalized

Variables

- Variable names should be short but descriptive (e.g., a variable for the mean of `x` should be named `mean_x`)
  - We are using `x` for pedagogical purposes... In practice, `x` is likely non-descriptive!
- When a variable name contains multiple words (e.g., apple size), there are a couple of approaches:
  - Use an underscore (e.g., `apple_size`)
  - Use "camel case" (e.g., `appleSize`)
    - Camel case has the first word all lowercase, with subsequent words having their first letter capitalized
- If a variable is not meant to change, it is called a constant. These variables are usually named in all-caps, e.g.:

```
PI = 3.14
```

Variables

Important

- Variable names cannot contain spaces (hence the use of camelCase and underscores)

Variables

! Important

- Variable names cannot contain spaces (hence the use of camelCase and underscores)
- Variable names cannot start with a number

Variables

Important

- Variable names cannot contain spaces (hence the use of camelCase and underscores)
- Variable names cannot start with a number
- Avoid naming variables with names that have already been "taken"
  - E.g., do not name a variable "`print`"
  - We will encounter more names that have been "taken"

Variables

! Important

- Variable names cannot contain spaces (hence the use of camelCase and underscores)
- Variable names cannot start with a number
- Avoid naming variables with names that have already been "taken"
  - E.g., do not name a variable "`print`"
  - We will encounter more names that have been "taken"
- Do not capitalize the first letter of a variable unless it is a `class` (beyond the scope of this workshop)

Variables

Arithmetic

```
a = 1
b = 2

print(a + b)     # * for multiplication, / for division
```

```
3
```

Note

- Use # to make a **comment**

Variables

Updating Variables

```
x = 10
x = 20
x = 15

print(x)
```

```
15
```

Variable Types

1. `int` (integer; "whole number")
   - less memory
   - 1 is an `int`
2. `float` (real; "decimal number")
   - 1.2 is a `float`
   - 1.0 is a `float`
3. `str` (string; characters/words with no numerical meaning)
   - `"Hello world!"` is a string
   - `"416-676-6767"` is also a string
4. `bool` (boolean; can be `True` or `False`; more on this later)
5. `None`
   - A special type of variable that essentially means "nothing meaningful is stored"
   - Useful as a placeholder

Variable Types

Checking Type

```
x = 416
print(x, type(x))    # type() to check type
```

```
416 <class 'int'>
```

Converting Type

```
x = 416
x = str(x)       # str() to convert to string
print(x, type(x))
```

```
416 <class 'str'>
```

- Can also use `int()` and `float()` to convert to respective type

A Brief Note

- `print()`, `type()`, `str()`, etc. are all examples of **functions**
  - These are essentially pre-made chunks of code for you to use
  - More on this later!

What will the below code output?

```
x = "Variable"

print(type(x))
print(x)
```

Section 3

More Advanced Variable Types

Lists

- Multiple values can be stored in one variable. A list is one way to do this:

Defining Lists

```
x = [0, 1, 2, 3, 4]      # Define a list; square brackets
print(type(x))
```

```
<class 'list'>
```

Lists: Indexing

```
x = [0,1,2,3,4]
```

- To extract one or more values from a list, you **index** the list
- Python uses **zero-based indexing**
  - This means that the first element of a list is index 0, the second element is index 1, ...
  - Element x is index $x - 1$

Lists: Indexing

Indexing Lists

```
x = [0, 1, 2, 3, 4]
y = 2

print(x[0])      # Index first element of a list
print(x[-1])     # Index last element
print(x[y])      # Index yth element
```

```
0
4
2
```

Lists: Indexing

- You can also take more than one element, or a **slice**, of the list

Slicing Lists

```
x = [0, 1, 2, 3, 4]

print(x[0:2])   # First and second (but not third) elements
```

```
[0, 1]
```

! Important

- `[inclusive:exclusive]`
  - The index on the *left-hand* side of the slice is *included*, whereas the index on the *right-hand* side of the slice is *excluded*

Lists: Indexing

Slicing Lists, continued

```
x = [0, 1, 2, 3, 4]

print(x[0:3]) # First three elements
print(x[0:1]) # Silly way to index just the first element
```

```
[0, 1, 2]
[0]
```

Lists: Length

- To find the length of a list:

len() function

```
x = [0, 1, 2, 3, 4]

print(len(x))   # The length of list x

# Another way to find the last element (not so silly!)
print(x[len(x) - 1])
```

```
5
4
```

Lists: Methods

- Lists have certain functions that can be applied to them, called **methods**
  - Methods are more general than this (e.g., string methods; more on these in a little bit!)

.append()

```
x = [0,1,2,3,4]

x.append(5)
print(x)
```

```
[0, 1, 2, 3, 4, 5]
```

- Adds an entry to a list

Lists: Methods

.extend()

```
x = [0,1,2,3,4]

x.extend([6,7,8,9])
print(x)
```

```
[0, 1, 2, 3, 4, 6, 7, 8, 9]
```

- Extend a list by "glueing" it to another list

Lists: Methods

Difference Between .append() and .extend()

```
x = [0,1,2,4]
y = [5,6,7,8]

x.append(y)
print(x)

x = [0,1,2,4]    # Redefine x
x.extend(y)
print(x)
```

```
[0, 1, 2, 4, [5, 6, 7, 8]]
[0, 1, 2, 4, 5, 6, 7, 8]
```

Lists: Changing Values

Changing a Value in a List

```
x = [0, 1, 2, 3, 4]

x[0] = 9
print(x)
```

```
[9, 1, 2, 3, 4]
```

Tuples

- Like lists, tuples contain multiple values
- Unlike lists, tuples are **immutable**; they cannot be changed once created.
  - Lists are *mutable*

Defining a Tuple

```
x = ("a", "tuple")  # Round brackets

print(x[0], x[1])
print(type(x))
```

```
a tuple
<class 'tuple'>
```

Tuples

Changing Values... ERROR!

```
x = ("a", "tuple")

x[0] = "Not a"
```

```
TypeError: 'tuple' object does not support item assignment
---------------------------------------------------------------
TypeError                                 Traceback (most rece
Cell In[427], line 3
      1 x = ("a", "tuple")
      2
----> 3 x[0] = "Not a"
TypeError: 'tuple' object does not support item assignment
```

Tuples

- Tuples have special properties, one of which is below:

Unpacking Tuples

```
x, y = (6, 7)

print(x)
print(y)
```

```
6
7
```

Section 4

Strings

Strings

- Strings are a lot like lists in that they are **iterable**
  - For now, think of these (lists, strings) as a sequence of values

Strings are Like Lists!

```
cringe = "Hello world!"

print(cringe[0:5])
print(len(cringe))
```

```
Hello
12
```

Strings

- We can join two strings together through **concatenation**

Concatenation

```
a = "Hello"
b = "World!"

c = a + " " + b      # Note the space between words
print(c)
```

```
Hello World!
```

Strings

- Sometimes you need to include quotations or other characters inside a string that would normally return an error
- This can be done through an **escape sequence**

ERROR! Why You Need Escape Sequences...

```
x = "This is a "string""
```

```
SyntaxError: invalid syntax (886282512.py, line 1)
  Cell In[431], line 1
    x = "This is a "string""
                   ^
SyntaxError: invalid syntax
```

Escape Sequences

```
x = "This is a \"string\""
print(x)
```

```
This is a "string"
```

Note

- The \ indicates the start of an escape sequence

Strings

- Strings are also kind of like tuples in that they are **immutable**

String Immutability

```
cringe = "Hello world!"
cringe[0:5] = "Goodbye"      # Error
```

```
TypeError: 'str' object does not support item assignment
```

```
---------------------------------------------------------------------
TypeError                                 Traceback (most rece
Cell In[433], line 2
      1 cringe = "Hello world!"
----> 2 cringe[0:5] = "Goodbye"      # Error
TypeError: 'str' object does not support item assignment
```

Strings

- We can turn to string **methods** to "modify" strings when needed (they just create a new string "behind the scenes")

.replace()

```
cringe = "Hello world!"

print(cringe.replace("Hello", "Goodbye"))
```

```
Goodbye world!
```

Strings

.split()

```
meme_list = "Harambe, 67, Jar Jar Binks, The Rizzler"

# Split on ", "; returns a list
new_list = meme_list.split(", ")

print(new_list)
print(new_list[0]) # Index the list
```

```
['Harambe', '67', 'Jar Jar Binks', 'The Rizzler']
Harambe
```

Strings

How might you index “The Rizzler”?

```
meme_list = "Harambe, 67, Jar Jar Binks, The Rizzler"
new_list = meme_list.split(", ")
```

Strings

How might you index “The Rizzler”?

```
meme_list = "Harambe, 67, Jar Jar Binks, The Rizzler"
new_list = meme_list.split(", ")
```

```
print(new_list[3])
print(new_list[-1])
print(new_list[len(new_list) - 1])
```

```
The Rizzler
The Rizzler
The Rizzler
```

Strings

- If you wanted to include variables within text, format strings as per the following:

f-string Example

```
x = 3.14159

print(f'Approximate value of pi is {x}')

print(f'To three decimal places is {x:.3f}')
```

```
Approximate value of pi is 3.14159
To three decimal places is 3.142
```

Strings

- The string (surrounded by either single `''` or double `""` quotations) is preceded by `f`
- The variable is surrounded by curly braces `{}` within the string
- Formatting is applied to the variable *within the curly braces*
  - A `:` signals that the proceeding formatting code should be applied to the variable

f-string Formatting Basics

- `x:.3f`
  - Format variable `x`
  - Round to 3 decimal places
  - `f` means `float`

Section 5

Booleans

Booleans

- A boolean variable (type `bool`) can be either `True` (`1`) or `False` (`0`).

Booleans

```
x = True
y = 10
print(type(x), type(y))

print(y > 5)
print(y < 5)
print(type(y < 5))
```

```
<class 'bool'> <class 'int'>
True
False
<class 'bool'>
```

Booleans

Booleans

```
x = 5
y = 10

print(x == 5)
print(x > 2 and y > 2)  # Both need to be True
print(x > 2 and y < 2)
print(x > 2 and not y < 2)
print(x > 2 or y < 2)   # At least one needs to be True
```

```
True
True
False
True
True
```

Booleans

Comparison Operators

- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to
- == equal to
- != not equal to

Logical Operators

- `and`
- `or`
- `not`

Section 6

Conditional Statements

Conditional Statements

- If thing1 happens, do x
- If thing2 happens instead, do y
- Otherwise, do z

Conditional Statements

- If thing1 happens, do x
- If thing2 happens instead, do y
- Otherwise, do z

Basic `if`/`elif`/`else`

```
x = 10  # Value presumably unknown
y = 20

if x == 4:
    print("x is 4")
elif x > y:
    print("x is bigger than y")
else:
    print("none of the above")
```

```
none of the above
```

Conditional Statements

Structure of Conditionals

```
if x == 4:
    print("x is 4")
```

- Begins with `if`/`elif` ("else if")/`else`
- Boolean expression/value
- Colon :
- Indented code underneath that runs if condition is met

Section 7

Loops

`while` Loops

- Loops are used to run a chunk of code repeatedly
  - Often a set number of times
- This is best shown and unpacked through examples:

while Loops

while Loop Basics

```
count = 1    # Initialize counter

while count <= 5:
    print(count)
    count = count + 1    # Increment counter
```

```
1
2
3
4
5
```

while Loops

Iterating through a list

```
x = [0,1,2,3]
count = 0

while count <= len(x) - 1:
    print(x[count])
    count += 1    # Same as count = count + 1
```

```
0
1
2
3
```

while Loops

- The `break` statement is used to exit a loop

break

```
x = 0

while True:      # Infinite loop...
    if x > 10:
        break    # Until you break!
    x += 1

print(x)
```

```
11
```

for Loops

- Often used with the `range()` function to loop over code a set number of times

for Loops, range()

```
for i in range(3):
    print(i)
```

```
0
1
2
```

for Loops

Iterating Over Entries in a List

```
x = [10,20,30,67]

for i in x:
    print(i)
```

```
10
20
30
67
```

for Loops

- Use `enumerate()` and a second indexing variable (usually `i` or `j`) if you want to keep variable position

enumerate()

```
x = [10,20,30,67]

for i, j in enumerate(x):
    print(i, j)
```

```
0 10
1 20
2 30
3 67
```

for Loops

- Loops (and conditionals) may be nested inside one another (like Russian dolls / Matryoshka)

Nesting

```
x = ["big", "small"]
y = ["cat", "dog"]

for i in x:
    for j in y:
        print(i, j)
```

```
big cat
big dog
small cat
small dog
```

Section 8

Importing Packages, Defining Functions

Importing Packages

- Packages add "extra stuff" to Python, including functions, constants, and more
- Some packages are installed with Python by default, and no extra work is required to use them

import math

```
import math

print(f'pi = {math.pi:.3f}')
```

```
pi = 3.142
```

Importing Packages

- We can "rename" packages to make their usage more concise

"Renaming" Packages

```
import math as m

print(f'pi = {m.pi:.3f}')
```

```
pi = 3.142
```

Importing Packages

- Some require installation. This is most easily (and safely) done through **Anaconda**
- Let's install the package **NumPy**!

Importing Packages

import numpy as np

```
import numpy as np       # Common convention

# Take a sample of 3 values from N(0, 1)
print(np.random.normal(loc = 0, scale = 1, size = 3))
```

```
[-0.36881847 -1.90636988 -0.09961063]
```

Note

- loc $= \mu$, scale $= \sigma$

Importing Packages

- A brief aside:
  - Let's say of all the functions in **NumPy**, you only care about `numpy.random.normal`

from ... import ...

```
from numpy.random import normal

# Same thing, but no `np.random` at the start
normal(loc = 0, scale = 1, size = 3)
```

```
array([ 1.6995373 , -0.38342312, -0.88985686])
```

Defining Functions

- Sometimes you repeat the same code over and over again
  - When this happens, it is useful to collapse the repeated code into a function
  - Otherwise, code becomes long and messy due to copy-pasting

Defining Functions

- Sometimes you repeat the same code over and over again
  - When this happens, it is useful to collapse the repeated code into a function
  - Otherwise, code becomes long and messy due to copy-pasting
  - For instance, imagine needing to know how to code your own `np.random.normal()` function and then copy pasting this code every time you want to use it

Defining Functions

Defining Basic Functions

```
def addition(a, b):
    return a + b                    # What the functions spits out

print(addition(a = 1, b = 5)) # Call function

def subtraction(a, b = 1):      # Give b default value of 1
    return a - b

print(subtraction(a = 5))
```

```
6
4
```

Defining Functions

Basic Structure of a Function

```
# "arg" short for "argument"
def function_name(arg1, arg2 = default_value, arg_n):
    return output
```

Defining Functions

Function Example

```
x = [0,1,2,3,4,5,6,7]

def qm_mean(things):
    sum = 0

    for i in things:
        sum += i

    return sum / len(things)

qm_mean(x)
```

```
3.5
```

Section 9

NumPy

NumPy

- Somewhat similar to MATLAB if you have used that
- Useful for working with arrays/matrices
- Stands for **Num**erical **Py**thon
- Based in computer language **C**
  - Faster, more efficient computations

NumPy

Working With Lists/Arrays

```
import numpy as np

list1 = [1,2,3,4]
print(list1 * 2)
print(type(list1))

numpyList = np.array(list1)  # Convert list -> numpy array
print(numpyList * 2)
print(type(numpyList))
```

```
[1, 2, 3, 4, 1, 2, 3, 4]
<class 'list'>
[2 4 6 8]
<class 'numpy.ndarray'>
```

NumPy

- Matrices are typically worked with as **2D** numpy arrays

2D NumPy Arrays

```
array2d = np.array([[1,2,3], [4,5,6]]) # List of lists
print(array2d)
print(type(array2d))
```

```
[[1 2 3]
 [4 5 6]]
<class 'numpy.ndarray'>
```

NumPy:

To Check Attributes of a NumPy Variable

```
array2d = np.array([[1,2,3],  # Matrices typed in a more
                    [4,5,6]]) # appealing manner

print(array2d.shape)     # Shape (# rows, # columns)
print(array2d.ndim)      # Number of dimensions
print(array2d.size)      # Number of elements
print(array2d.dtype)     # Type of elements
```

```
(2, 3)
2
6
int64
```

NumPy

- Some numpy types:
  - `uint8`
  - `int32`
  - `int64`
  - `float64`
- Generally, numpy type = type learned before + memory (in bits)

Note

- The `u` in `uint8` stands for "unsigned," meaning that the variable can only store positive values

np.arange()

```
np.arange(start=10, stop=20, step=2, dtype = "int64")
```

```
array([10, 12, 14, 16, 18])
```

Note

- `stop` parameter is *exclusive*

np.linspace()

```
np.linspace(start=10, stop=20, num=6, dtype = "int64")
```

```
array([10, 12, 14, 16, 18, 20])
```

Tip

- Use `np.linspace()` for breaking something into equal parts
- Use `np.arange()` for creating a sequence of numbers

NumPy

- NumPy has lots of functions relating to choosing random values

Random Choice

```
print(np.random.choice(["A", "B", "C", "D"]))

print(np.random.choice(["A", "B", "C", "D"], size = 2))
```

```
C
['D' 'B']
```

Random Numbers

```
print(np.random.normal(loc = 0, scale = 1, size = 3))
print(np.random.uniform(low = 0, high = 10, size = 3))
```

```
[ 0.1391333  -1.69384897 -0.12024384]
[1.10005531 7.99002154 4.97919112]
```

NumPy

- To make the values from these random functions reproducible, we set a "seed"
  - i.e., the first time the below `np.random.choice()` call is run will always return `'D'`, the second time `'B'`

Setting a Seed

```
np.random.seed(67)

# 1st time returns 'D'
print(np.random.choice(["A", "B", "C", "D"]))

# 2nd time returns 'B'
print(np.random.choice(["A", "B", "C", "D"]))
```

```
D
B
```

Indexing NumPy Arrays

```
x = np.random.normal(0, 10, size=(3,3))
print(x, "\n")  # Print x with a blank line after it

print(x[2,2], type(x[2,2]))
print(x[2,:])   # Row 2 (zero-based), all columns
```

```
[[  4.24396269   8.19276386  -4.55523498]
 [ -1.74348913 -12.91666321  -5.12083276]
 [  1.51374927   2.9382983  -14.96950941]]

-14.96950940608669 <class 'numpy.float64'>
[  1.51374927   2.9382983  -14.96950941]
```

Indexing (cont.)

```
x = np.random.normal(0, 10, size=(3,3))
print(x, "\n")

print(x[2, 0:2])     # Row 2, columns 0 and 1
print(type(x[2, 0:2]))
```

```
[[  5.54692736  -9.74439429  -5.77253404]
 [-12.7560138   -7.86514849  -0.27373561]
 [ -1.48018218  -5.27834347  -1.01387863]]

[-1.48018218 -5.27834347]
<class 'numpy.ndarray'>
```

NumPy

- You may also perform **aggregate** functions on numpy arrays

NumPy Aggregate Functions

```
x = np.array([1,4,7,15])

print(np.mean(x))
print(np.median(x))
print(np.std(x))
```

```
6.75
5.5
5.2141634036535525
```

More Aggregate Functions

```
x = np.array([1,4,7,15])

print(np.min(x))          # Minimum value
print(np.argmin(x))       # Position of minimum value
print(np.max(x))          # Maximum value
print(np.argmax(x))       # Position of maximum value
```

```
1
0
15
3
```

NumPy

Check-in

- How might you generate 100 random values from the standard normal distribution?
- How might you get the maximum of these values?

NumPy

Check-in

- How might you generate 100 random values from the standard normal distribution?
- How might you get the maximum of these values?

```
np.max(np.random.normal(loc=0, scale=1, size=100))
```

```
np.float64(1.6230497183839991)
```

Section 10

pandas

pandas

- For the analysis of spreadsheet-like data in Python
- Somewhat similar to R if you have used that
  - Introduces the data frame

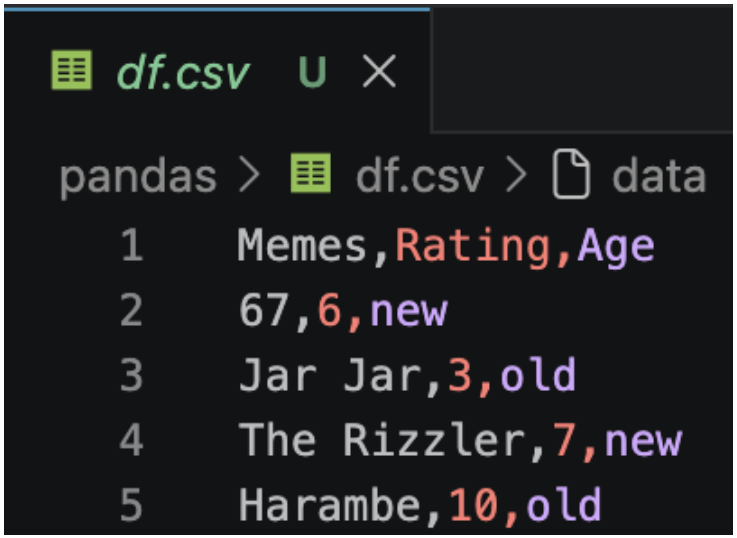
Creating a Data Frame

```
import pandas as pd
df = pd.DataFrame({
    "Memes" : ["67", "Jar Jar", "The Rizzler", "Harambe"],
    "Rating": [6, 3, 7, 10],
    "Age"   : ["new", "old", "new", "old"]
}, index = ['a','b','c','d']) # Index is optional
df # To print
```

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old
c	The Rizzler	7	new
d	Harambe	10	old

pandas

- Data are commonly stored as `.csv` files
  - **C**omma **s**eparated **v**alues
- These files have entries separated by commas, often including a first row of "header" entries
- A `.csv` file of the pandas data frame from the previous slide (variable `df`) is shown below



pandas

- Continuing with the `df` data frame, the below shows how a `.csv` file is written

Writing `.csv` Files

```
# Write df.csv to the pandas/ folder
df.to_csv("pandas/df.csv", index = False)
```

- Here, `to_csv` is a **method** acting on the `df` **object** (or variable)

Tip

Setting the `index` argument to `False` removes row indices (i.e., row 0, 1, 2, ..., n) from being stored as a separate column

pandas

- We often need to do the reverse; namely, reading data from a `.csv` file into a pandas data frame

Reading `.csv` Files

```
# Read df.csv from the pandas/ folder
df_new = pd.read_csv("pandas/df.csv")

df_new
```

	Memes	Rating	Age
0	67	6	new
1	Jar Jar	3	old
2	The Rizzler	7	new
3	Harambe	10	old

pandas

- pandas also has functions to read/write data from other file formats, such as from Microsoft Excel `.xlsx` files

`.xlsx` Files

```
excelDat = pd.read_excel( # Read
    "file_name.xlsx", sheet_name = "optional"
)

excelDat.to_excel( # Write
    "folder/excelDat.xlsx", sheet_name = "optional"
)
```

Inspecting Data

```
df.head(2) # First 2 rows
```

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old

```
df.tail(2) # Last 2 rows
```

	Memes	Rating	Age
c	The Rizzler	7	new
d	Harambe	10	old

Inspecting Data

```
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 4 entries, a to d
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Memes   4 non-null      str
 1   Rating  4 non-null      int64
 2   Age     4 non-null      str
dtypes: int64(1), str(2)
memory usage: 128.0+ bytes
```

Descriptives

```
df.describe()
```

	Rating
count	4.000000
mean	6.500000
std	2.886751
min	3.000000
25%	5.250000
50%	6.500000
75%	7.750000
max	10.000000

Grouping

```
df.groupby("Age").describe()
```

	Rating							
	count	mean	std	min	25%	50%	75%	max
Age								
new	2.0	6.5	0.707107	6.0	6.25	6.5	6.75	7.0
old	2.0	6.5	4.949747	3.0	4.75	6.5	8.25	10.0

Selecting a Column

```
df["Memes"]
```

```
a              67
b         Jar Jar
c     The Rizzler
d         Harambe
Name: Memes, dtype: str
```

A Note on Types

```
type(df["Memes"])
```

`pandas.Series`

- A variable of type `pandas.Series` is essentially a column of a pandas data frame

- To select multiple columns, provide a list of column names as the index

Selecting Multiple Columns

```
df[["Memes", "Age"]]
```

	Memes	Age
a	67	new
b	Jar Jar	old
c	The Rizzler	new
d	Harambe	old

- A simple way to filter rows is by row number:

Filter Rows by Row Number

```
df[0:2] # Row numbers 0 and 1
```

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old

- You can also filter rows by indexing a data frame with a boolean expression involving the same data frame

Filter Rows With Boolean Expressions

```
df[df["Age"] == "old"]
```

	Memes	Rating	Age
b	Jar Jar	3	old
d	Harambe	10	old

- The result of the previous slide was also a data frame!
  - This means we can index it further by, say, taking the `"Memes"` column:

Indexing Twice

```
df[df["Age"] == "old"]["Memes"]
```

```
b    Jar Jar
d    Harambe
Name: Memes, dtype: str
```

- This also means that methods shown previously can be applied to these filtered data frames

Describing Filtered Data

```
df[df["Age"] == "old"].describe()
```

	Rating
count	2.000000
mean	6.500000
std	4.949747
min	3.000000
25%	4.750000
50%	6.500000
75%	8.250000
max	10.000000

- We can also select rows and columns using the `loc` or `iloc` **attributes**
  - These are like methods but use square `[]` brackets (instead of round brackets `()`)

Selecting with `loc` and `iloc`

```
df.loc["b", "Memes"] # Row b of Memes column
```

```
'Jar Jar'
```

```
df.iloc[1,0] # Row 1 of 0th column (Memes)
```

```
'Jar Jar'
```

- Like **numpy**, we can use a colon (`:`) with `loc` and `iloc` to indicate "all rows" or "all columns":

Selecting All Rows with `loc`

```
df.loc[:, "Memes"] # All rows of Memes column
```

```
a              67
b         Jar Jar
c     The Rizzler
d         Harambe
Name: Memes, dtype: str
```

Selecting All Columns with `iloc`

```
df.iloc[0] # All columns for 0th row (row 'a')
```

```
Memes      67
Rating      6
Age       new
Name: a, dtype: object
```

Note

- If only the row argument is supplied, all columns are selected by default
  - This is also true for `loc`

- If you would like to select multiple columns (or rows), but not all columns (or rows), pass a `list` to `loc` or `iloc`

Selecting Multiple (Specific) Columns with `loc`

```
df.loc[["a", "b"], ["Memes", "Age"]]
```

	Memes	Age
a	67	new
b	Jar Jar	old

- Same thing for `iloc`, but with row numbers instead of indices/names

- We can also use boolean expressions with `loc` (but not `iloc`):

Boolean Expressions with `loc`

```
# Take rows where age is old and just the meme column
df.loc[df["Age"] == "old", "Memes"] # Old memes!
```

```
b    Jar Jar
d    Harambe
Name: Memes, dtype: str
```

- In these boolean expressions, we can use `&` for `and`, `|` for `or`, and `~` for `not`
  - We need these special operators as we are comparing entire columns as opposed to single values
  - We also surround expressions to be compared in round brackets `()`

Boolean Expressions with `loc`

```
# Take all rows where age is old and rating is larger than 7
df.loc[(df["Age"] == "old") & (df["Rating"] > 7), :]
```

	Memes	Rating	Age
d	Harambe	10	old

	Memes	Rating	Age
a	67	6	new
b	Jar Jar	3	old
c	The Rizzler	7	new
d	Harambe	10	old

How might I index rows with rating greater than or equal to seven?

- I want 2 different answers! (Hint: `loc` and `df[?]`)

How might I index rows with rating greater than or equal to seven?

```
df[df["Rating"] >= 7]
df.loc[df["Rating"] >= 7, :] # The `, :` is optional
```

	Memes	Rating	Age
c	The Rizzler	7	new
d	Harambe	10	old

- To show a peculiarity of pandas, I first set the index of our data frame `df` (e.g., the row names `"a"`, `"b"`, ...) back to integers (the default)

Changing the Index

```
df.index = [0,1,2,3]
df
```

	Memes	Rating	Age
0	67	6	new
1	Jar Jar	3	old
2	The Rizzler	7	new
3	Harambe	10	old

- When using `loc` on a data frame with a numeric index, the format of a slice is `inclusive:inclusive`

`loc` is Strange

```
df.loc[0:1] # inclusive:inclusive
```

	Memes	Rating	Age
0	67	6	new
1	Jar Jar	3	old

- For reference, here is the same line but using `iloc` instead of `loc`:

`iloc` is Typical

```
df.iloc[0:1] # inclusive:exclusive
```

	Memes	Rating	Age
0	67	6	new

- We can also add columns to our data frame. Here I add a column `r10`: the `Rating` out of 10

Adding a Column

```
df["r10"] = df["Rating"] / 10
df
```

	Memes	Rating	Age	r10
0	67	6	new	0.6
1	Jar Jar	3	old	0.3
2	The Rizzler	7	new	0.7
3	Harambe	10	old	1.0

- We might also want to sort these entries by `Rating`

Sorting a Data Frame

```
df.sort_values(by = "Rating", inplace = True)
df
```

	Memes	Rating	Age	r10
1	Jar Jar	3	old	0.3
0	67	6	new	0.6
2	The Rizzler	7	new	0.7
3	Harambe	10	old	1.0

- Argument `inplace = True` modifies `df` directly instead of returning a new one

- We can also sort by `Age` first and then `Rating`
  - Strings are sorted alphabetically

Sorting by a List

```
df.sort_values(by = ["Age", "Rating"], inplace = True)
df
```

	Memes	Rating	Age	r10
0	67	6	new	0.6
2	The Rizzler	7	new	0.7
1	Jar Jar	3	old	0.3
3	Harambe	10	old	1.0

pandas

- Instead of using the `describe()` method, you may want only a select few (or one) statistic

Getting Specific Descriptive Statistics

```
df["Rating"].mean()
```

```
np.float64(6.5)
```

```
df["Rating"].std()
```

```
np.float64(2.886751345948129)
```

Getting Specific Descriptive Statistics

```
df["Rating"].agg(['mean', 'std']) # agg = aggregate
```

```
mean    6.500000
std     2.886751
Name: Rating, dtype: float64
```

Section 11

matplotlib.pyplot

matplotlib.pyplot

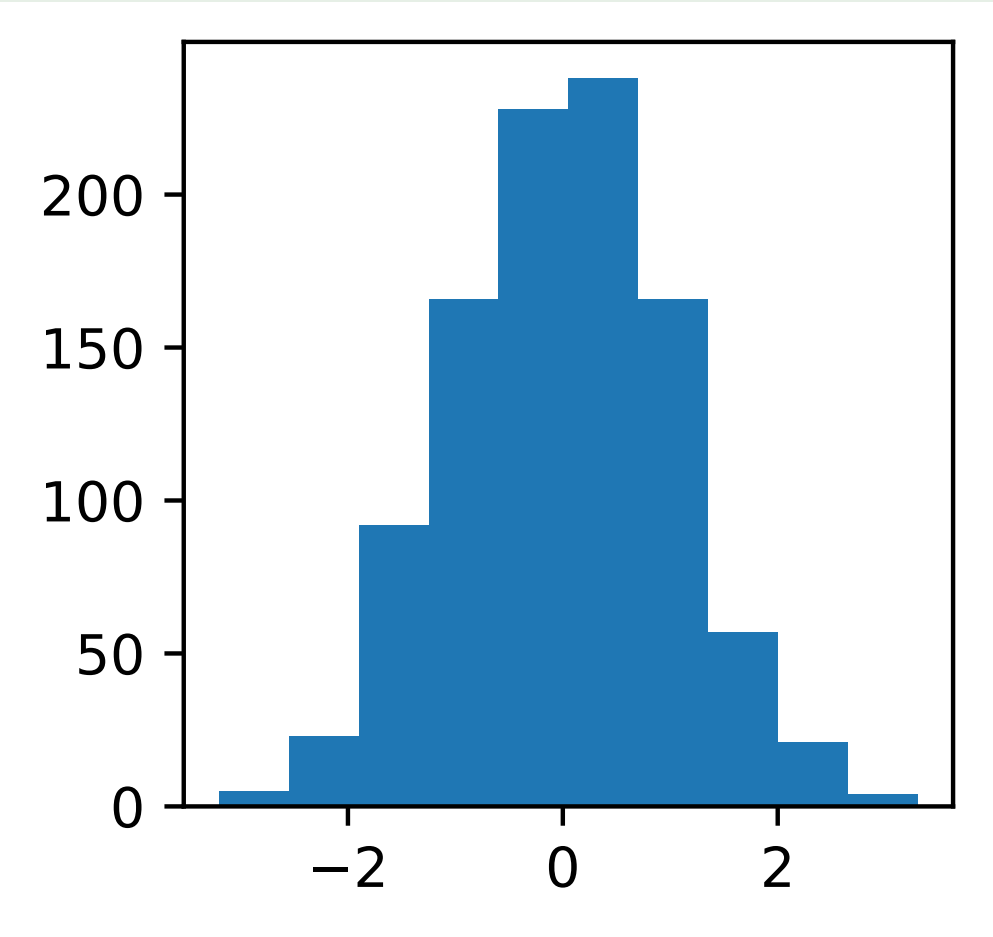
- **matplotlib.pyplot** is a package for visualizing data in Python
- Fun fact: If you have asked ChatGPT for a graph, you likely received one that was made using this package

Basic Histogram

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.normal(0, 1, 1000)
plt.hist(x)
plt.show()
```

Basic Histogram

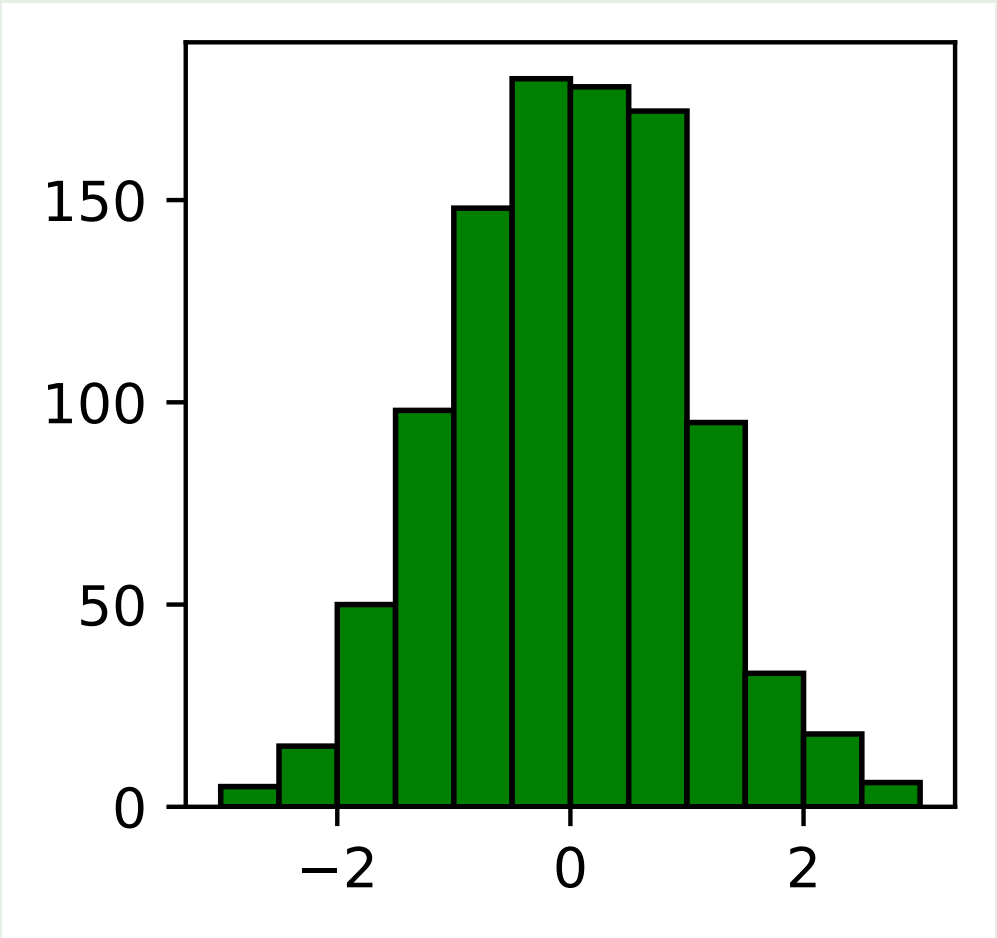


- A few ways to modify the histogram are illustrated below

Modifying the Plot

```
plt.hist(
    x,                          # Data
    color = "green",            # Bar Colour
    edgecolor = "black",        # Outline Colour
    bins = np.arange(-3, 3.5, 0.5) # Bins of 0.5 from -3 to +3
)
plt.show()
```

Modifying the Plot

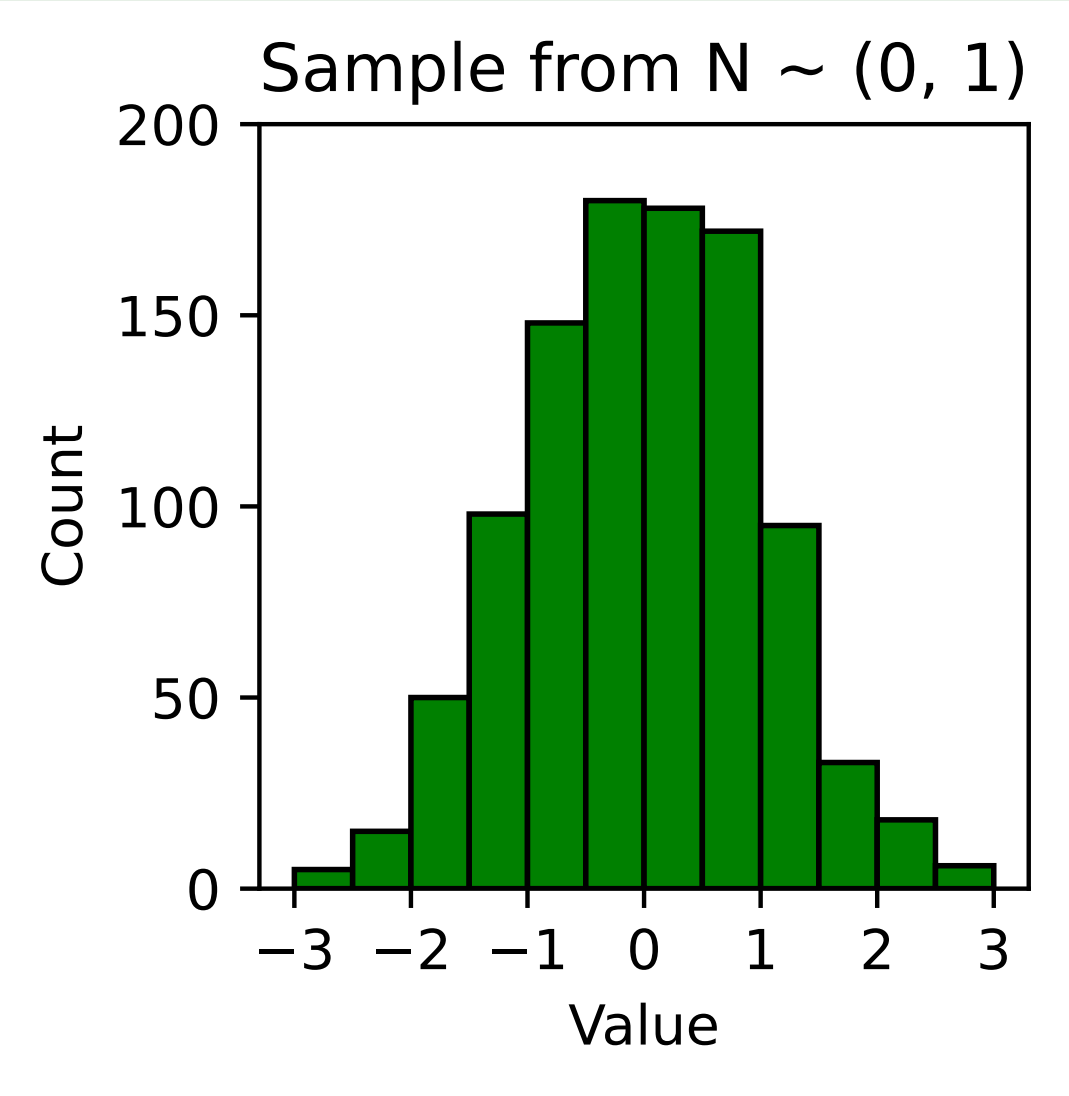


More Ways to Modify

```
plt.hist(
    x, color = "green", edgecolor = "black",
    bins = np.arange(-3, 3.5, 0.5)
)

plt.xticks(np.arange(-3, 4, 1)) # x-axis ticks spaced by 1
plt.title("Sample from N ~ (0, 1)")
plt.xlabel("Value")
plt.ylabel("Count")
plt.show()
```

More Ways to Modify



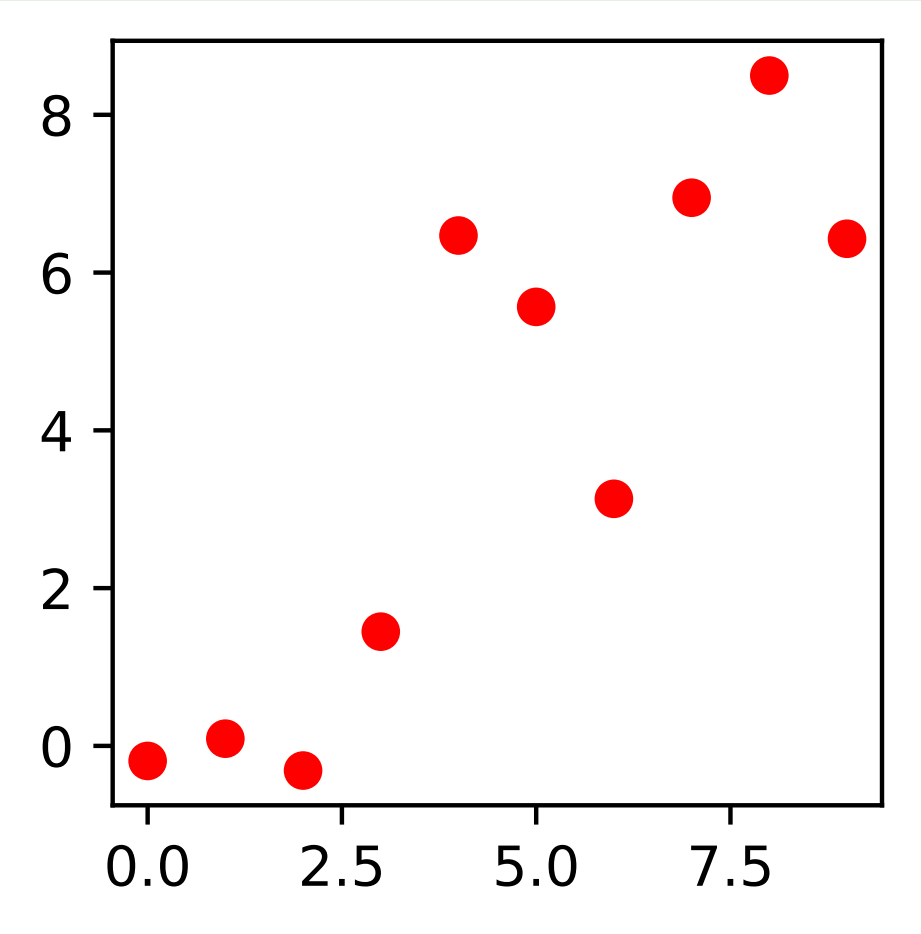
- The below is how data was generated for subsequent scatterplot examples

Simulated Data for a Simple Regression

```
errors = np.random.normal(0, 2, 10) # Normal errors
nums_x = np.arange(0, 10, 1) # Nums 0 to 9
nums_y = nums_x + errors # y = x + errors
```

Scatterplot

```
plt.plot(nums_x, nums_y, 'ro')  # plot(x, y, style)
plt.show()                      # 'ro' = red circles
```



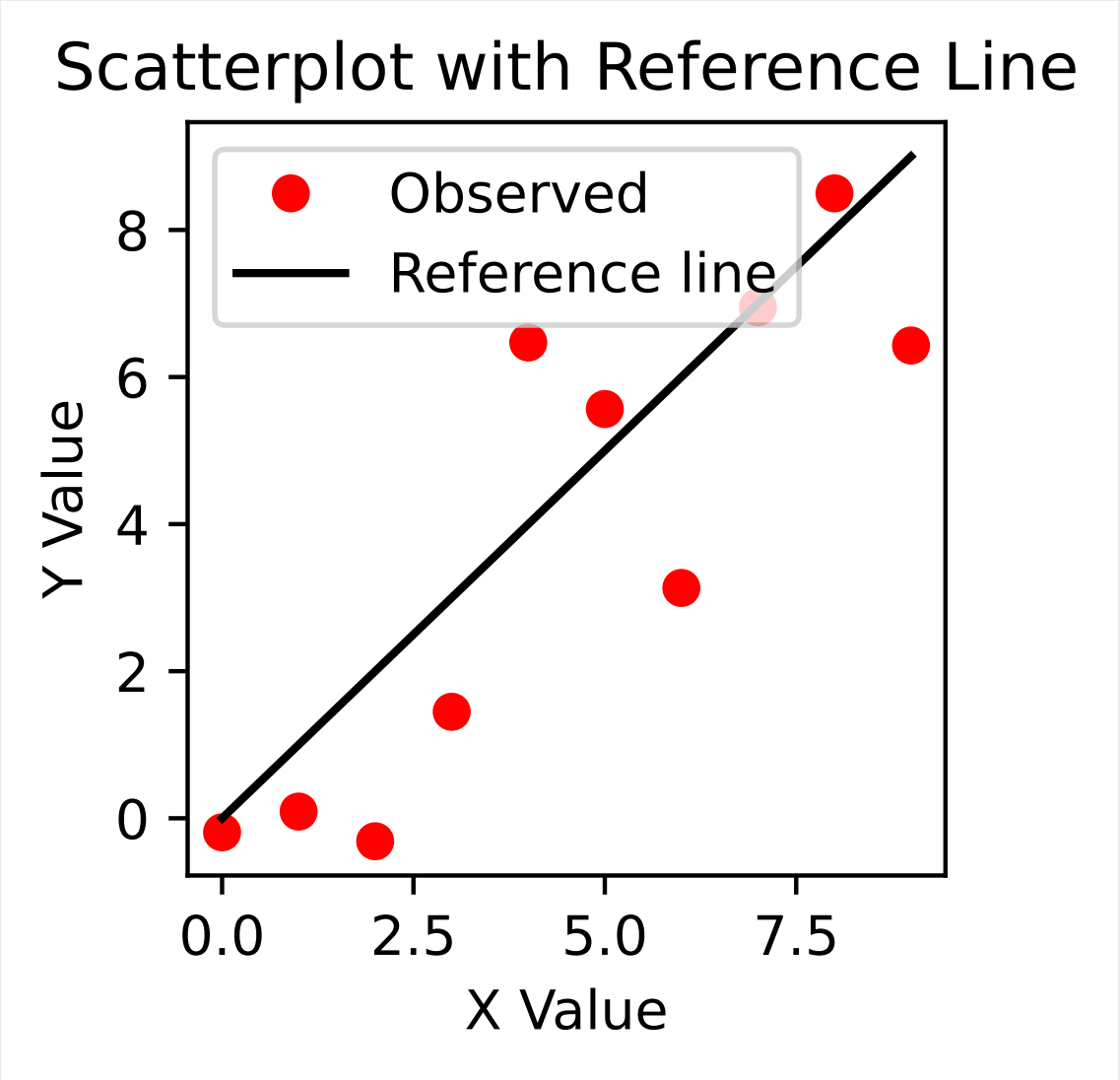
matplotlib.pyplot

- You can add another plot layer by calling another plotting function before `plt.show()`
- This is useful for adding:
  - reference lines
  - fitted lines
  - multiple groups

Adding a Line to a Scatterplot

```
plt.plot(nums_x, nums_y, 'ro')        # Observed data: red circ
plt.plot(nums_x, nums_x, 'k-')        # Reference line: black s
plt.show()
```

Adding Labels and a Legend



- `label` gives each layer a name

Section 12

statsmodels

statsmodels

- `statsmodels` is a package for statistical testing and modelling
- It is useful when you want:
  - formal hypothesis tests
  - regression models
  - p-values and confidence intervals
  - detailed statistical output

Importing `statsmodels` Functions

```
import statsmodels.formula.api as smf
from statsmodels.stats.weightstats import ttest_ind
```

statsmodels

- `statsmodels.formula.api` contains functions that use formula notation
- It is commonly renamed `smf`
- `ttest_ind` is a function for independent-samples t-tests

Import Structure

```
import statsmodels.formula.api as smf
#      package.module          alias

from statsmodels.stats.weightstats import ttest_ind
#    package.module.submodule        function
```

Statistical Testing

- Statistical tests help us use a **sample** to make inferences about a larger population
- A test usually starts with a **null hypothesis**
  - Often "no difference" or "no relationship"
- We compare the observed data to what would be expected if the null hypothesis were true

Note

- The code runs the test, but the researcher chooses whether the test makes sense

Statistical Testing

General Workflow

1. State the research question
2. Identify the variable types and study design
3. Summarize and visualize the data
4. Run the appropriate test
5. Interpret the estimate, confidence interval, and p-value

Example Data

- We will use simulated scores from two independent groups
- Each row represents one participant
- The group labels are stored in a pandas DataFrame

Creating the Data

```
import pandas as pd

np.random.seed(42)

scores_df = pd.DataFrame({
    "group": ["control"] * 30 + ["treatment"] * 30,
    "score": np.concatenate([
        np.random.normal(70, 10, 30),
        np.random.normal(76, 10, 30)
    ])
})
```

Example Data

Inspecting the Data

```
print(scores_df.head())    # First five rows
print(scores_df.shape)     # (number of rows, number of columns
```

```
     group      score
0  control  74.967142
1  control  68.617357
2  control  76.476885
3  control  85.230299
4  control  67.658466
(60, 2)
```

- Always inspect the data before running a test

Example Data

Group Summary

```
print(
    scores_df.groupby("group").agg(
        n = ("score", "count"),
        mean = ("score", "mean"),
        sd = ("score", "std")
    )
)
```

```
                n       mean        sd
group
control        30  68.118531  9.000064
treatment      30  74.788375  9.311022
```

- `groupby("group")` summarizes the control and treatment rows separately
- Each tuple specifies `(column, summary function)`

Independent-Samples t-Test

- Used to compare the means of **two independent groups**
- Example: Do treatment and control groups have different mean scores?
- Do not use this test if the same participants appear in both groups

Null hypothesis:

$$H_0 : \mu_{\text{treatment}} - \mu_{\text{control}} = 0$$

Independent-Samples t-Test

Selecting Each Group

```
control = scores_df.loc[
    scores_df["group"] == "control",  # Rows to include
    "score"                           # Column to return
]

treatment = scores_df.loc[
    scores_df["group"] == "treatment",
    "score"
]
```

- `ttest_ind()` needs the two groups as separate variables

Welch's t-Test

- Welch's t-test is an independent-samples t-test
- It does **not** assume equal population variances
- It is a useful default when comparing two independent groups

Running the Test

```
test_result = ttest_ind(
    treatment,        # First group
    control,          # Second group
    usevar = "unequal"
)

print(test_result)
```

```
(np.float64(2.82107476851771), np.float64(0.006544738502602257
```

ttest_ind()

Function Structure

```
ttest_ind(
    x1,                     # Values from the first group
    x2,                     # Values from the second group
    alternative = "two-sided",
    usevar = "unequal",   # Welch's t-test
    value = 0               # Difference under the null hypothesi
)
```

- Use `alternative = "two-sided"` when either group could have the larger mean
- Use `value = 0` when the null hypothesis is no mean difference

ttest_ind()

- ttest_ind() returns three values:
  1. t-statistic
  2. p-value
  3. degrees of freedom

Tuple Unpacking

```
t_statistic, p_value, degrees_freedom = ttest_ind(
    treatment,
    control,
    usevar = "unequal"
)

print(t_statistic)
print(p_value)
print(degrees_freedom)
```

```
2.8210747685177
0.006544738502602257
```

t-Test Interpretation

- The t-statistic describes the group difference relative to its uncertainty
- The p-value describes how incompatible the result is with the null hypothesis
- The degrees of freedom help determine the t-distribution used by the test

! Important

- A p-value is **not** the probability that the null hypothesis is true

Mean Difference

- The t-test output should be interpreted alongside the observed group difference

Calculating the Mean Difference

```
mean_difference = treatment.mean() - control.mean()

print(f"Treatment mean: {treatment.mean():.2f}")
print(f"Control mean: {control.mean():.2f}")
print(f"Mean difference: {mean_difference:.2f}")
```

```
Treatment mean: 74.79
Control mean: 68.12
Mean difference: 6.67
```

Reporting a t-Test

Formatting the Result

```
print(
    f"Mean difference = {mean_difference:.2f}, "
    f"t({degrees_freedom:.2f}) = {t_statistic:.2f}, "
    f"p = {p_value:.3f}"
)
```

```
Mean difference = 6.67, t(57.93) = 2.82, p = 0.007
```

- Report the observed difference and p-value together
- Use cautious language such as “evidence of a difference”

Correlation

- A correlation describes the direction and strength of a **linear relationship**
- Pearson's correlation coefficient is represented by r
- It ranges from -1 to 1

Important

- Correlation does not establish causation

Correlation

Adding a Second Variable

```
np.random.seed(7)

scores_df["study_hours"] = (
    2
    + 0.08 * scores_df["score"]
    + np.random.normal(0, 1.5, len(scores_df))
)
```

Correlation

Scatterplot

```
plt.plot(scores_df["study_hours"], scores_df["score"], "bo")
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.show()
```

- Use a scatterplot before calculating a correlation

Correlation

.corr() Method

```
correlation = scores_df["study_hours"].corr(
    scores_df["score"]
)

print(f"r = {correlation:.2f}")
```

```
r = 0.53
```

- .corr() describes the observed sample relationship
- Regression can test whether the population slope differs from zero

Simple Linear Regression

- Simple linear regression examines the relationship between:
  - one continuous **outcome**
  - one continuous **predictor**
- Here, the outcome is `score` and the predictor is `study_hours`

$$\text{score} = \text{intercept} + (\text{slope} \times \text{study hours}) + \text{error}$$

Formula Notation

Formula Structure

```
"score ~ study_hours"
# outcome ~ predictor
```

- The outcome goes on the left of ~
- The predictor goes on the right of ~
- Column names must match the DataFrame

smf.ols()

Creating a Model

```
model_definition = smf.ols(
    formula = "score ~ study_hours",
    data = scores_df
)

print(type(model_definition))
```

```
<class 'statsmodels.regression.linear_model.OLS'>
```

- ols stands for **ordinary least squares**
- smf.ols() defines the model but does not estimate its coefficients yet

.fit()

Fitting a Model

```
model = model_definition.fit()

print(type(model))
```

```
<class 'statsmodels.regression.linear_model.RegressionResultsW
```

- `.fit()` estimates the intercept, slope, and other model statistics
- Store the fitted result so its attributes can be inspected

Method Chaining

Defining and Fitting in One Step

```
model = smf.ols(
    "score ~ study_hours",
    data = scores_df
).fit()
```

- This is a common way to define and fit a model

.params

Model Parameters

```
print("Intercept:", round(model.params["Intercept"], 2))
print("study_hours:", round(model.params["study_hours"], 2))
```

```
Intercept: 50.43
study_hours: 2.75
```

- `Intercept`: predicted score when study hours equals zero
- `study_hours`: predicted score change for one additional study hour

.params

Accessing One Parameter

```
intercept = model.params["Intercept"]
slope = model.params["study_hours"]

print(f"Intercept: {intercept:.2f}")
print(f"Slope: {slope:.2f}")
```

```
Intercept: 50.43
Slope: 2.75
```

- Use the parameter name to extract a specific estimate

.pvalues

Testing the Slope

```
slope_p_value = model.pvalues["study_hours"]

print(f"study_hours p-value: {slope_p_value:.3f}")
```

```
study_hours p-value: 0.000
```

Null hypothesis:

$$H_0 : \text{population slope} = 0$$

.conf_int()

Confidence Intervals

```
slope_ci = model.conf_int().loc["study_hours"]

print(f"Lower: {slope_ci[0]:.2f}")
print(f"Upper: {slope_ci[1]:.2f}")
```

```
Lower: 1.59
Upper: 3.90
```

- A confidence interval gives a range of values compatible with the model and data

.summary().tables[0]

Model Overview

```
summary = model.summary()
print(summary.tables[0])
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  score   R-squared:                       0.280
Model:                            OLS   Adj. R-squared:                  0.268
Method:                 Least Squares   F-statistic:                     22.57
Date:                Wed, 17 Jun 2026   Prob (F-statistic):           1.37e-05
Time:                        21:15:58   Log-Likelihood:                -210.99
No. Observations:                  60   AIC:                             426.0
Df Residuals:                      58   BIC:                             430.2
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
```

- Table 0 summarizes the model setup and overall fit
- Useful for checking the outcome, sample size, R^2, and model type

.summary().tables[1]

Coefficient Table

```
print(summary.tables[1])
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      50.4278      4.553     11.076      0.000      41.314      59.541
study_hours     2.7466      0.578      4.751      0.000       1.589       3.904
==============================================================================
```

- Table 1 gives the estimates for each model term
- Focus first on `coef`, `P>|t|`, and `[0.025, 0.975]`

.summary().tables[2]

Diagnostics Table

```
print(summary.tables[2])
```

```
================================================================================
Omnibus:                        0.940   Durbin-Watson:                   1.864
Prob(Omnibus):                  0.625   Jarque-Bera (JB):                0.843
Skew:                          -0.281   Prob(JB):                        0.656
Kurtosis:                       2.852   Cond. No.                         34.0
================================================================================
```

- Table 2 contains diagnostic and assumption-related information
- Useful later when checking whether the model is behaving reasonably

.rsquared

- R-squared is the proportion of outcome variance described by the fitted model
- It ranges from 0 to 1
- A larger R-squared does not prove that the model is correct or causal

R-Squared Attribute

```
print(f"R-squared: {model.rsquared:.3f}")
```

```
R-squared: 0.280
```

.predict()

Predicted Outcomes

```
scores_df["predicted_score"] = model.predict(scores_df)

for i in range(3):
    print(
        f"Observed: {scores_df['score'][i]:.2f}; "
        f"Predicted: {scores_df['predicted_score'][i]:.2f}"
    )
```

```
Observed: 74.97; Predicted: 79.36
Observed: 68.62; Predicted: 69.08
Observed: 76.48; Predicted: 72.86
```

- .predict() calculates the fitted outcome for each row

Plotting Model Predictions

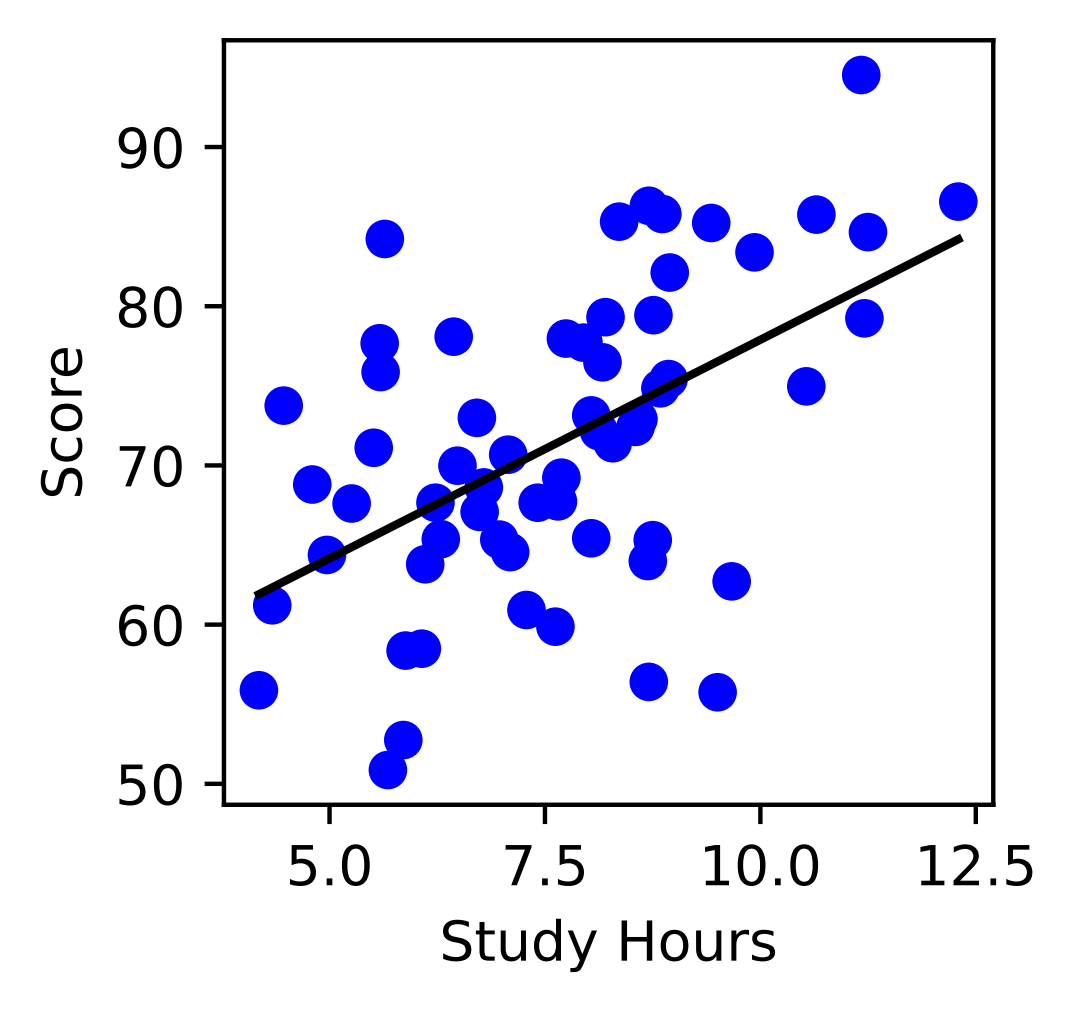
Observed and Predicted Values

```
ordered = scores_df.sort_values("study_hours")

plt.plot(scores_df["study_hours"], scores_df["score"], "bo")
plt.plot(ordered["study_hours"], ordered["predicted_score"], "
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.show()
```

Plotting Model Predictions

Fitted Line



- Blue circles are observed values
- The black line contains model-predicted values

Reporting Regression Results

Formatting the Result

```
slope_ci = model.conf_int().loc["study_hours"]

print(
    f"b = {model.params['study_hours']:.2f}, "
    f"95% CI [{slope_ci[0]:.2f}, {slope_ci[1]:.2f}], "
    f"p = {model.pvalues['study_hours']:.3f}"
)
```

```
b = 2.75, 95% CI [1.59, 3.90], p = 0.000
```

- Report the estimated slope, confidence interval, and p-value together
- Use **associated with**, rather than **caused**